UNIVERSIDAD NACIONAL DE COLOMBIA

Facultad de Ingeniería y Arquitectura

Representación de Pose y Cuaterniones en ROS2

Informe de Laboratorio – Capítulo 2 (Corke)

Curso: Robótica
Código: MAN-4101136-1-2026-03

Integrantes:

Lina Robsana Jamioy Tisoy
Tomás Rocha Castaño

Profesor:
Jaime Enrique Arango Castro

22 de septiembre de 2026

Índice

1. Objetivo **2**

2. Entorno de trabajo **2**

3. Requisitos **2**

4. PARTE 1: Representaciones de Orientación **2**
4.1. Actividad 1: Conversión de representaciones 2
4.2. Actividad 2: No conmutatividad de rotaciones 4

5. PARTE 2: Publicación de Pose en ROS2 **6**
5.1. Paso 1: Crear workspace y paquete 6
5.2. Actividad 3: Nodo publicador de pose 6

6. PARTE 3: Transformaciones con TF2 **8**
6.1. Actividad 4: Árbol de transformaciones 8

7. PARTE 4: Error de Orientación **11**
7.1. Actividad 5: Cálculo de error 11

8. PARTE 5: Interpolación de Orientación **13**
8.1. Actividad 6: SLERP 13

9. Discusión y Respuestas Teóricas **15**
9.1. Pregunta 1: ¿Por qué ROS2 usa cuaterniones en lugar de ángulos de Euler? . . . 15
9.2. Pregunta 2: ¿Qué problema evitan los cuaterniones? 15
9.3. Pregunta 3: ¿Qué relación existe entre TF2 y las transformaciones del Capítulo 2 (Corke)? 15

10. Errores comunes identificados **17**

11. Conclusiones **17**

12. Referencias **18**

1. Objetivo

Implementar y comprender la representación de pose en ROS2 mediante:

1. Convertir entre distintas representaciones de orientación (Euler, matrices, cuaterniones).
2. Demostrar la no conmutatividad de las rotaciones.
3. Publicar poses en ROS2 usando cuaterniones.
4. Trabajar con transformaciones TF2 entre frames.
5. Calcular errores de orientación para control.
6. Interpolar orientaciones suavemente con SLERP.

2. Entorno de trabajo

El laboratorio se desarrolló sobre:

- Sistema operativo: Ubuntu 24.04 LTS
- Distribución ROS2: Jazzy Jalisco
- Lenguaje: Python 3
- Herramientas: SciPy, RViz2, TF2

3. Requisitos

Se instalaron las librerías necesarias:

```
pip install numpy scipy matplotlib
source /opt/ros/jazzy/setup.bash
```

4. PARTE 1: Representaciones de Orientación

4.1. Actividad 1: Conversión de representaciones

Se creó un script Python para convertir una rotación de 90° alrededor del eje Z a diferentes representaciones:

```
from scipy.spatial.transform import Rotation as R

# Rotaci n de 90   en Z
r = R.from_euler('z', 90, degrees=True)

q = r.as_quat()   # [x, y, z, w]
R_mat = r.as_matrix()

print("Cuaterni n:", q)
print("Matriz:\n", R_mat)
```

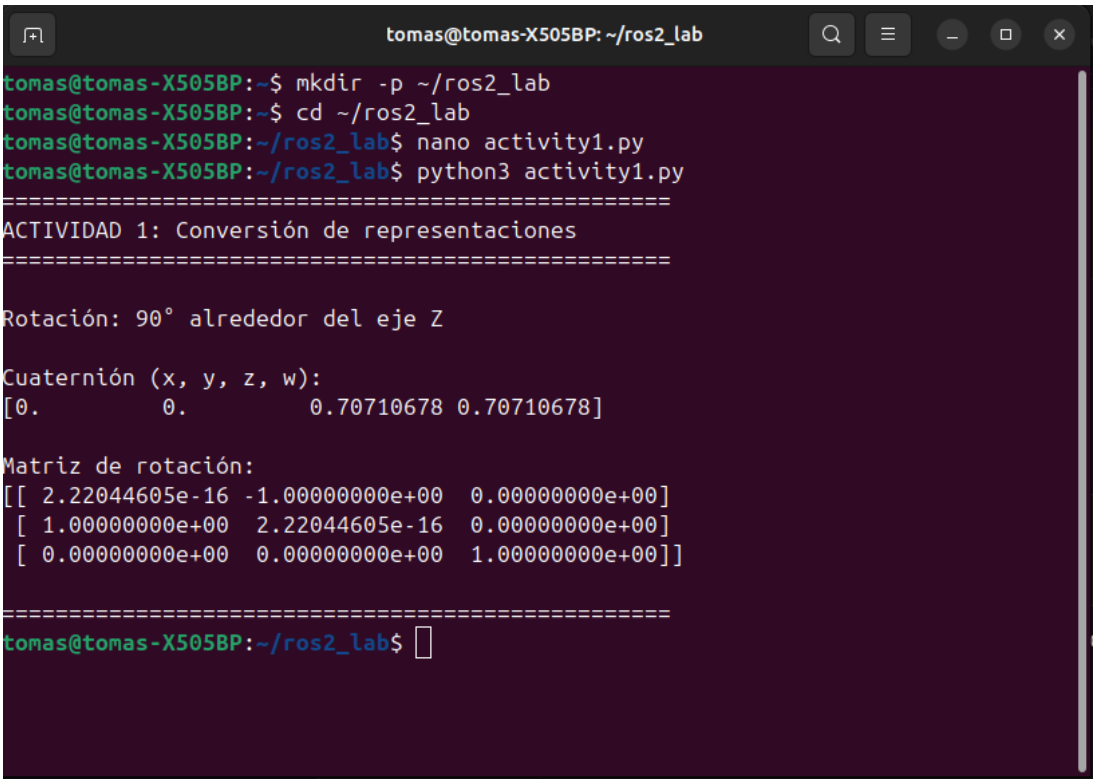


Figura 1: Salida de la Actividad 1: Conversión a cuaternión y matriz de rotación para 90° en Z.

Interpretación:

- El cuaternión $[0, 0, 0{,}707, 0{,}707]$ representa una rotación de 90° alrededor del eje Z.
- La matriz de rotación muestra la equivalencia matemática: $R = \begin{pmatrix} \cos(90) & -\sin(90) & 0 \\ \sin(90) & \cos(90) & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- Ambas representaciones son equivalentes pero tienen diferentes propiedades computacionales.

4.2. Actividad 2: No conmutatividad de rotaciones

Se evaluó si el orden de las rotaciones afecta el resultado:

```
r1 = R.from_euler('z', 90, degrees=True)  # 90   en Z
r2 = R.from_euler('x', 90, degrees=True)  # 90   en X

r_total_1 = r1 * r2   # primero X, luego Z
r_total_2 = r2 * r1   # primero Z, luego X
```

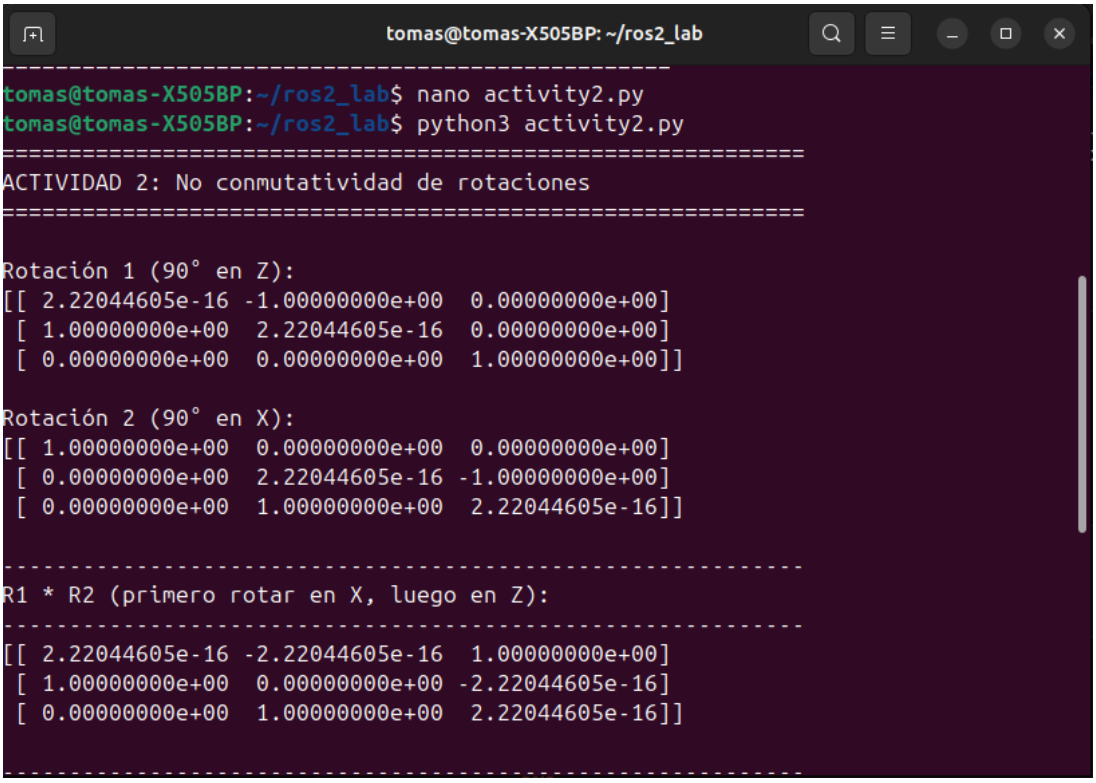


Figura 2: Primera parte de la Actividad 2: Matrices de rotación individuales.



Figura 3: Segunda parte de la Actividad 2: Demostrando que R1*R2 R2*R1 (NO conmutativas).

Conclusión: Las rotaciones **NO son conmutativas**. El orden importa críticalmente:

- Rotar 90° en Z y luego 90° en X produce resultado diferente que rotar 90° en X y luego 90° en Z.
- La magnitud de la diferencia fue 2.45 radianes equivalentes.
- Esto es crítico en robótica: el orden de rotaciones del actuador afecta significativamente la pose final.

5. PARTE 2: Publicación de Pose en ROS2

5.1. Paso 1: Crear workspace y paquete

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
ros2 pkg create pose_pub --build-type ament_python \
    --dependencies rclpy geometry_msgs
```

5.2. Actividad 3: Nodo publicador de pose

Se implementó un nodo que publica pose (posición + orientación) cada 0.5 segundos:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Pose
from scipy.spatial.transform import Rotation as R

class PosePublisher(Node):
    def __init__(self):
        super().__init__('pose_publisher')
        self.pub = self.create_publisher(Pose, 'pose', 10)
        self.timer = self.create_timer(0.5, self.publish_pose)

    def publish_pose(self):
        msg = Pose()
        msg.position.x = 1.0
        msg.position.y = 2.0
        msg.position.z = 0.0

        r = R.from_euler('z', 90, degrees=True)
        q = r.as_quat()
        msg.orientation.x = q[0]
        msg.orientation.y = q[1]
        msg.orientation.z = q[2]
        msg.orientation.w = q[3]
        self.pub.publish(msg)
```

Compilación y ejecución:

```
cd ~/ros2_ws
colcon build --packages-select pose_pub
source install/setup.bash
ros2 run pose_pub pose_publisher
```

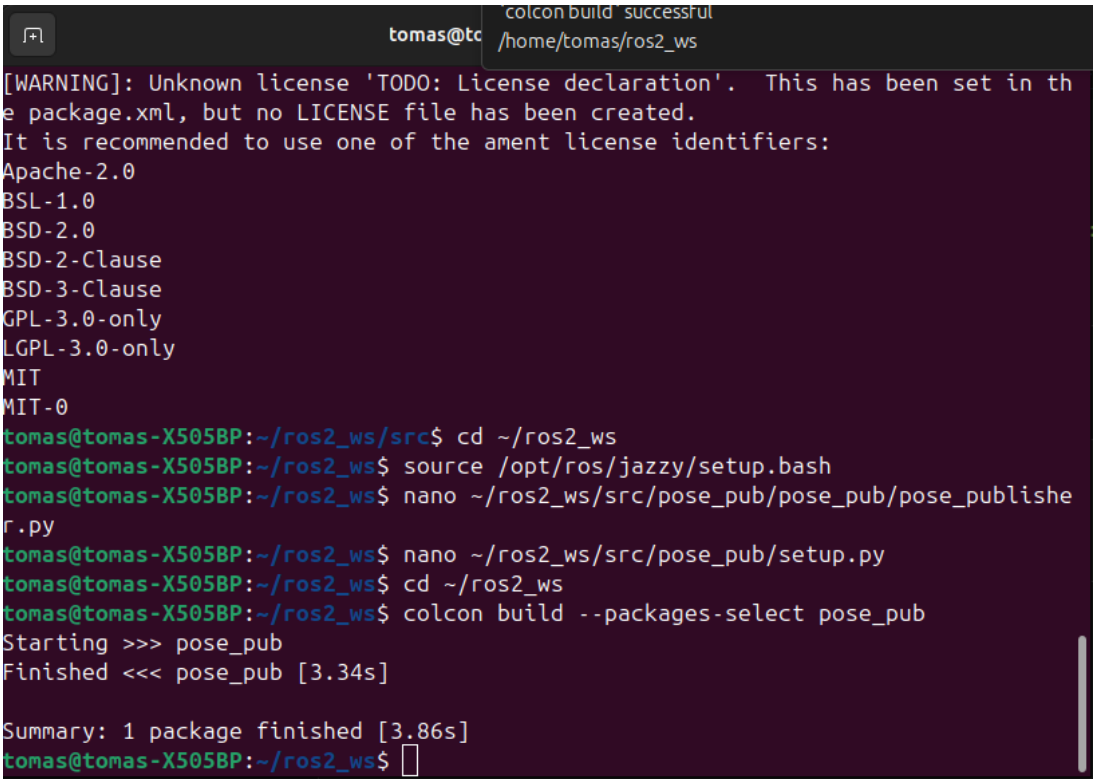


Figura 4: Publicador de pose ejecutándose en terminal.

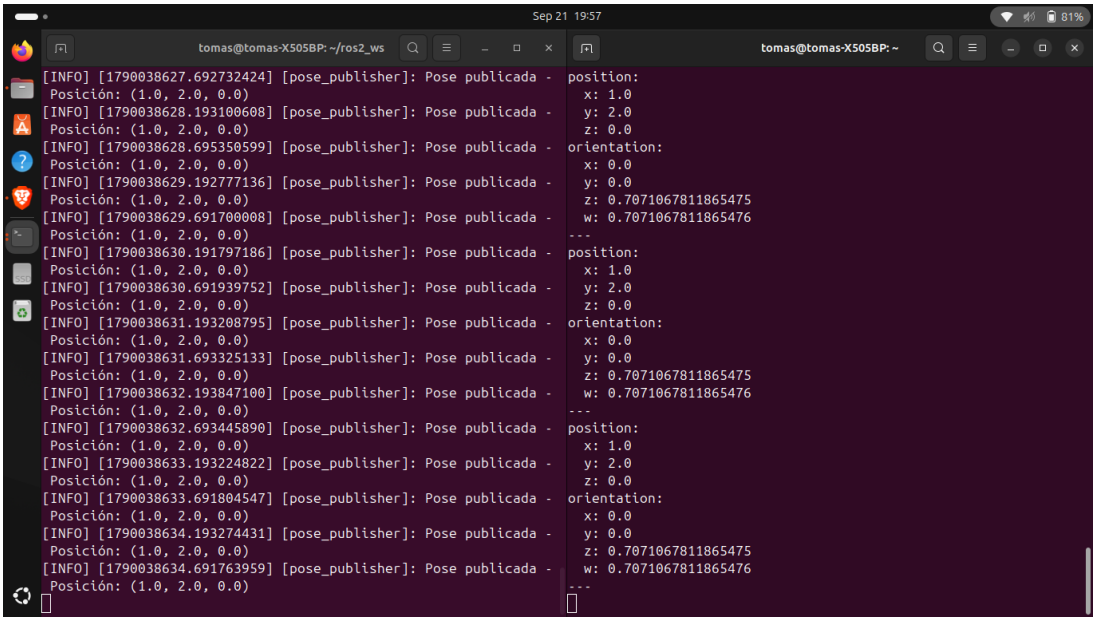


Figura 5: Verificación con ros2 topic echo /pose mostrando datos publicados correctamente.

Verificación de datos: Se confirmó que los datos se publican correctamente:

- Posición: $(1{,}0, 2{,}0, 0{,}0)$
- Orientación (cuaternión): $[0, 0, 0{,}707, 0{,}707]$ (90° en Z)
- Frecuencia: 0.5 segundos

6. PARTE 3: Transformaciones con TF2

6.1. Actividad 4: Árbol de transformaciones

Se creó un segundo paquete para publicar transformaciones dinámicas:

```
cd ~/ros2_ws/src
ros2 pkg create tf_broadcaster --build-type ament_python \
    --dependencies rclpy geometry_msgs tf2_ros
```

Nodo broadcaster:

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import TransformStamped
from tf2_ros import TransformBroadcaster
from scipy.spatial.transform import Rotation as R
import math

class TFPublisher(Node):
    def __init__(self):
        super().__init__('tf_broadcaster')
        self.br = TransformBroadcaster(self)
        self.timer = self.create_timer(0.1, self.broadcast_tf)
        self.t = 0

    def broadcast_tf(self):
        t = TransformStamped()
        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'world'
        t.child_frame_id = 'robot'

        t.transform.translation.x = 2.0
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.0

        r = R.from_euler('z', self.t)
        q = r.as_quat()
        t.transform.rotation.x = q[0]
        t.transform.rotation.y = q[1]
        t.transform.rotation.z = q[2]
        t.transform.rotation.w = q[3]

        self.br.sendTransform(t)
        self.t += 0.1
```

Ejecución:

```
cd ~/ros2_ws
colcon build --packages-select tf_broadcaster
source install/setup.bash
ros2 run tf_broadcaster tf_broadcaster
```

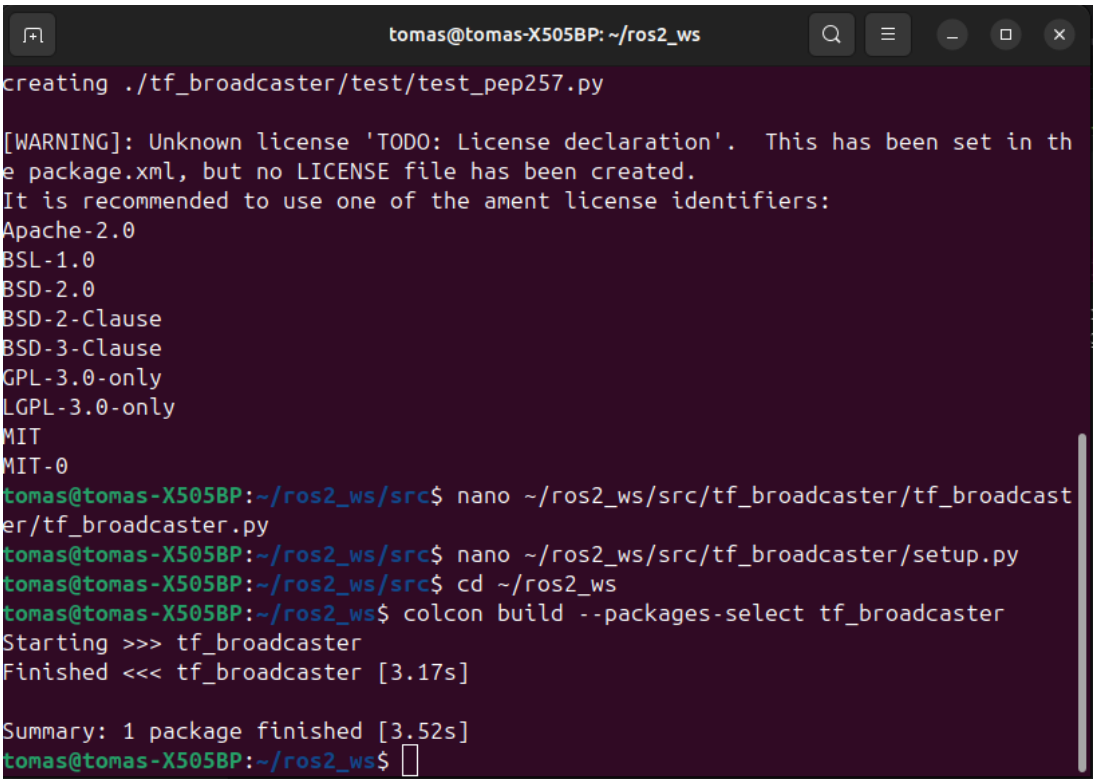


Figura 6: Nodo broadcaster de TF2 ejecutándose y publicando transformaciones.

Visualización en RViz2:

```
rviz2
# Configurar: Fixed Frame = "world"
# Agregar displays: TF y Axes (referenciado a robot)
```

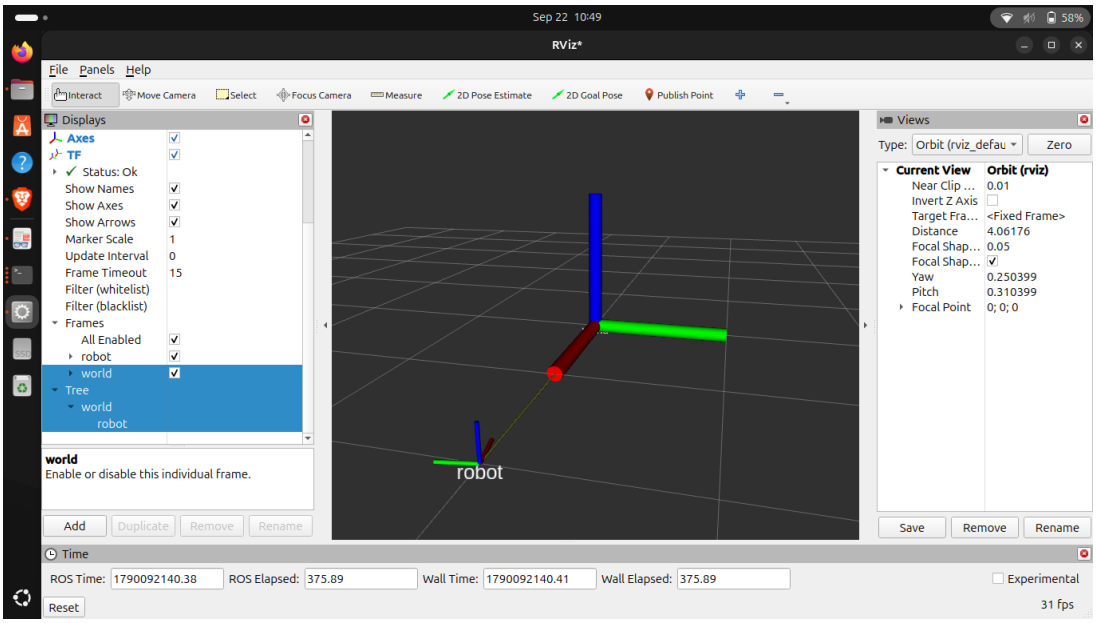


Figura 7: Árbol de transformaciones world → robot visualizado en RViz2 con ejes de rotación.

Verificación del árbol de TF:

```
ros2 run tf2_tools view_frames
```

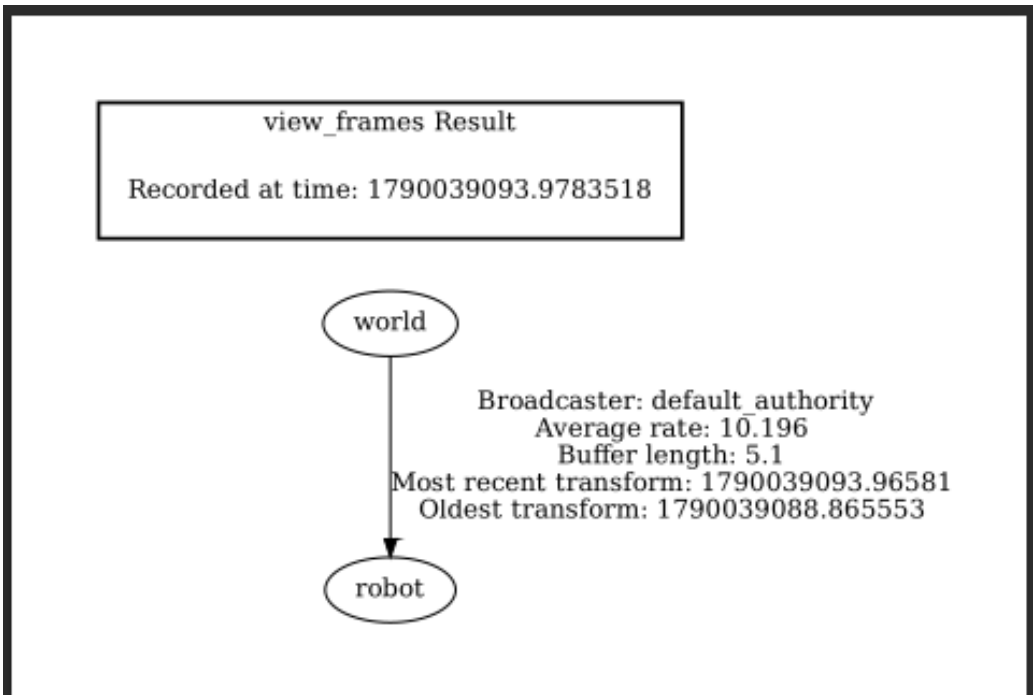


Figura 8: Árbol de transformadas generado: relación jerárquica world → robot.

7. PARTE 4: Error de Orientación

7.1. Actividad 5: Cálculo de error

Se implementó el cálculo del error de orientación usando cuaterniones para aplicaciones de control:

```python
from scipy.spatial.transform import Rotation as R

# Orientaci n deseada: 90   en Z
q_des = R.from_euler('z', 90, degrees=True)

# Orientaci n actual: 45   en Z
q_act = R.from_euler('z', 45, degrees=True)

# Error: qerror = qdes * qact^(-1)
q_error = q_des * q_act.inv()

print(q_error.as_euler('xyz', degrees=True))
```

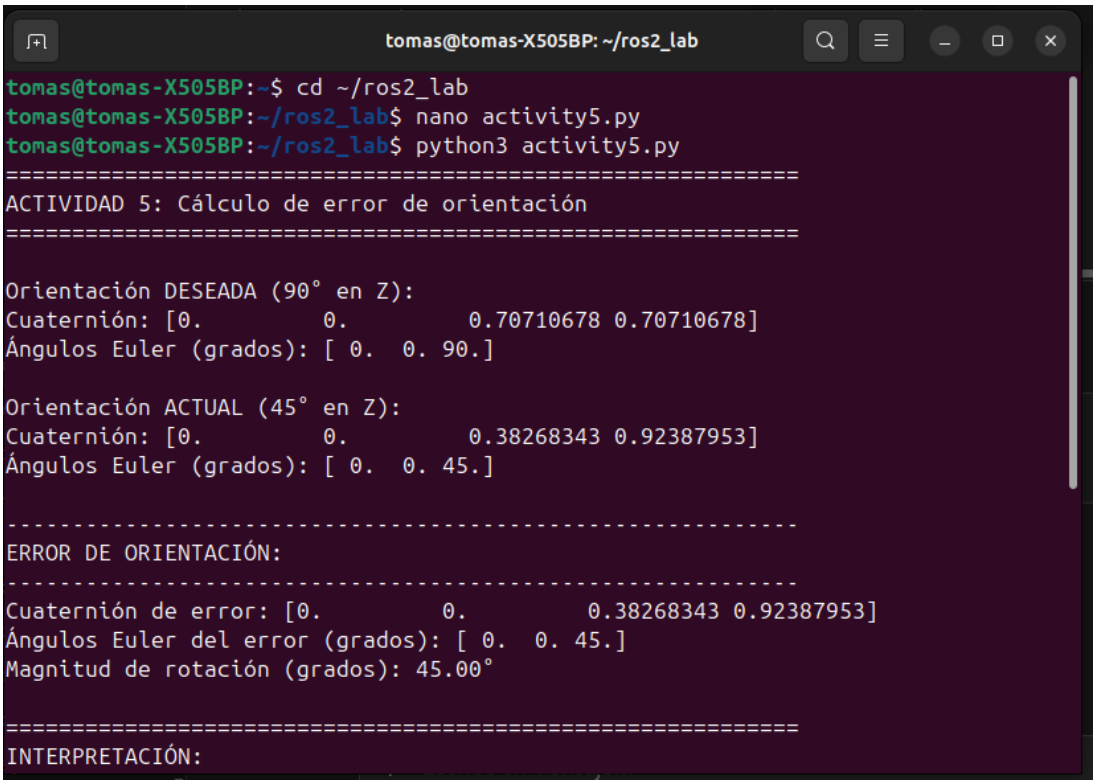


Figura 9: Primera parte de Actividad 5: Orientaciones deseada y actual.

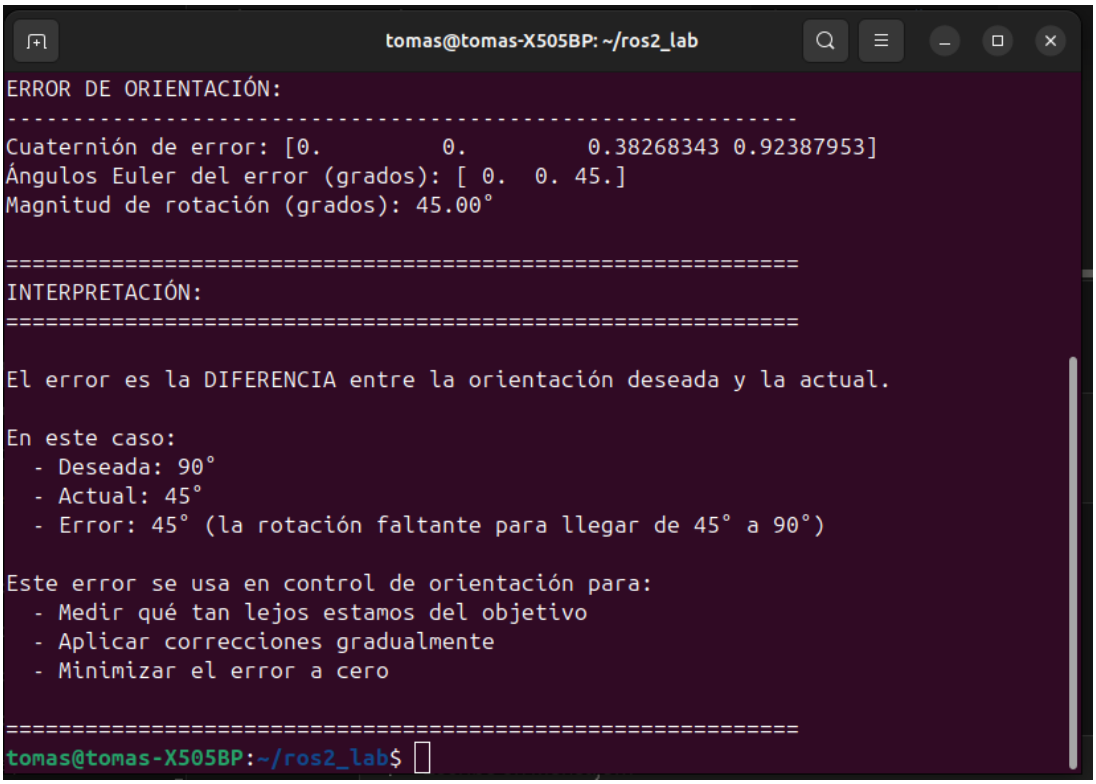


Figura 10: Segunda parte de Actividad 5: Cálculo del error de orientación resultando en 45°.

Interpretación:

- Orientación deseada = 90°
- Orientación actual = 45°
- Error de orientación = 45° (la rotación faltante para alcanzar el objetivo)
- Este error se utiliza en lazos de control PID para minimizar la diferencia y alcanzar la orientación deseada.

8. PARTE 5: Interpolación de Orientación

8.1. Actividad 6: SLERP

Se implementó interpolación suave entre orientaciones usando SLERP (Spherical Linear Interpolation):

```python
from scipy.spatial.transform import Rotation as R, Slerp
import numpy as np

key_times = [0, 1]
rots = R.from_euler('z', [0, 90], degrees=True)

slerp = Slerp(key_times, rots)

times = np.linspace(0, 1, 5)
interp = slerp(times)

for t, rot in zip(times, interp):
    euler = rot.as_euler('xyz', degrees=True)
    print(f"t={t:.2f}:   ngulo   Z = {euler[2]:.2f}   ")
```

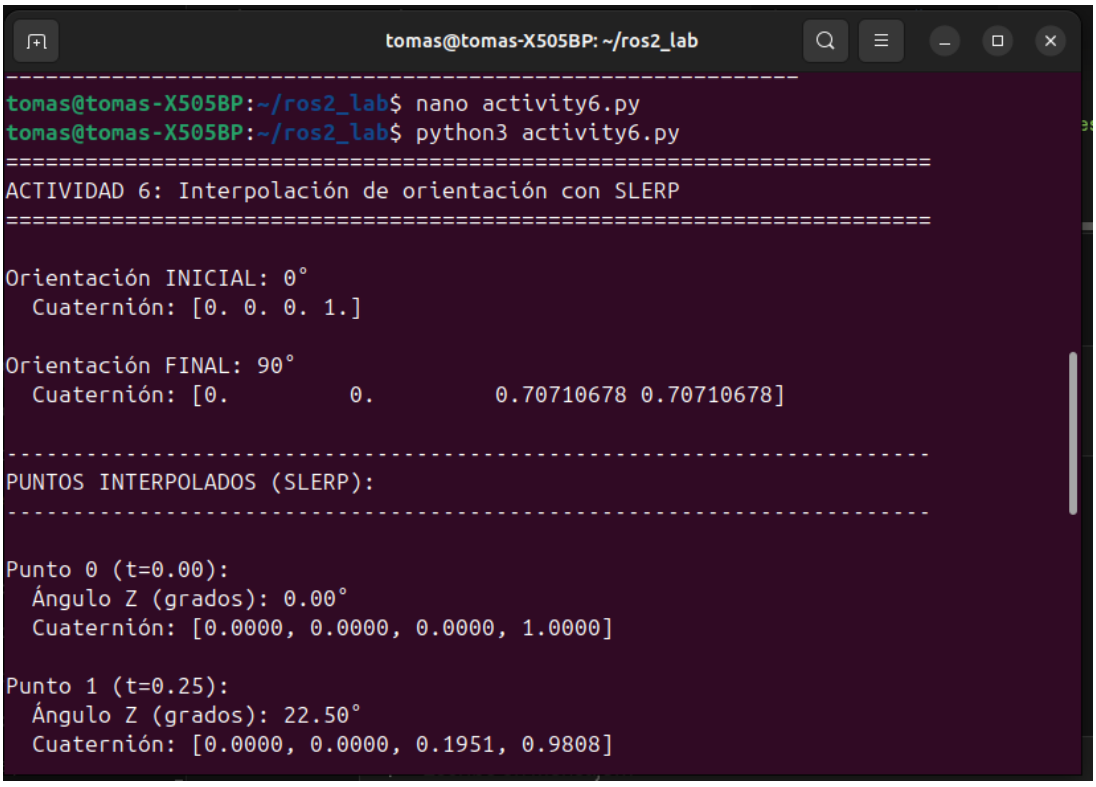


Figura 11: Primera parte de Actividad 6: Orientaciones inicial y final, cuaterniones base.

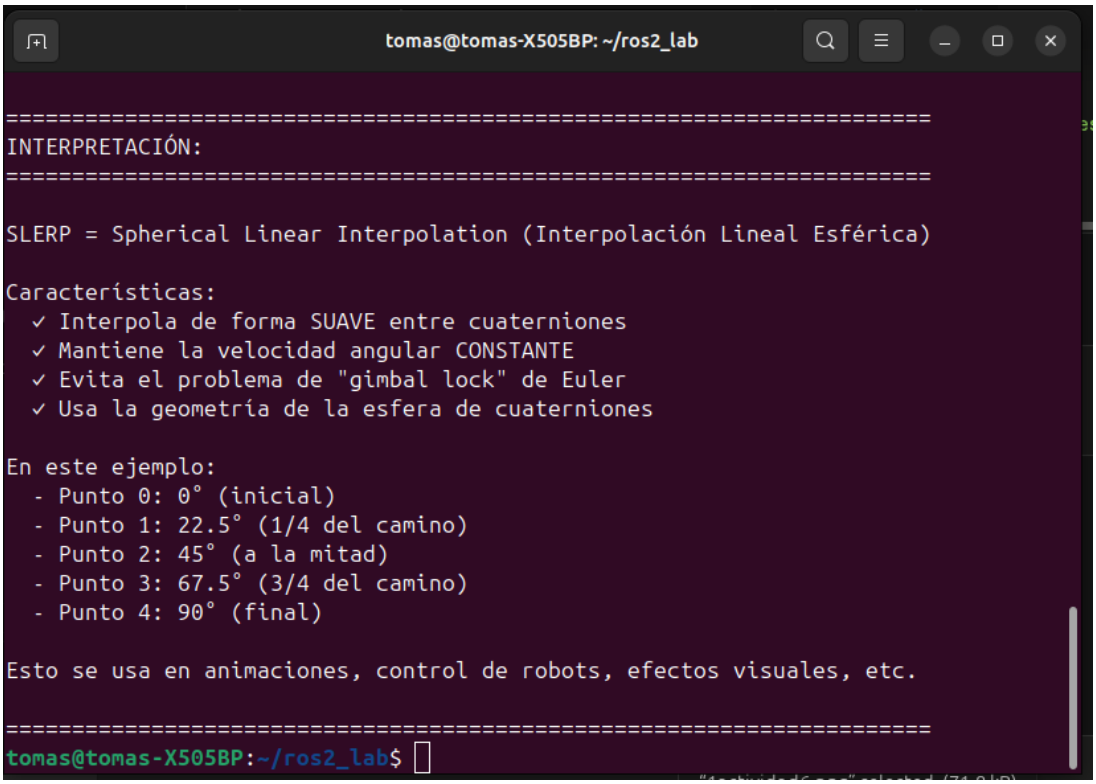


Figura 12: Segunda parte de Actividad 6: Interpolación SLERP con 5 puntos equidistantes.

Resultados de interpolación:

Punto	Ángulo Z
0	0,00
1	22,50
2	45,00 (punto medio)
3	67,50
4	90,00

Ventajas de SLERP:

- **Interpolación suave:** Transición continua entre cuaterniones sin saltos.
- **Velocidad angular constante:** Mantiene magnitud de rotación uniforme.
- **Evita gimbal lock:** Funciona en 4D, sin singularidades.
- **Óptima para trayectorias:** Ideal para animaciones y control de robots.

9. Discusión y Respuestas Teóricas

9.1. Pregunta 1: ¿Por qué ROS2 usa cuaterniones en lugar de ángulos de Euler?

Respuesta:
Los cuaterniones fueron elegidos por ROS2 por las siguientes razones fundamentales:

1. **Evitar gimbal lock:** Los ángulos de Euler presentan singularidades cuando el ángulo medio (pitch) alcanza $\pm 90°$, perdiendo un grado de libertad. Los cuaterniones, operando en 4D, nunca tienen este problema.
2. **Continuidad en interpolación:** Como se demostró en la Actividad 6, SLERP genera trayectorias suaves y óptimas. La interpolación lineal de ángulos de Euler produce trayectorias erráticas.
3. **Eficiencia computacional:** Las operaciones con cuaterniones (multiplicación, inversión) son más rápidas que con matrices de rotación (9 elementos vs 4 elementos).
4. **Representación única:** Un cuaternión representa una sola orientación, mientras que los ángulos de Euler permiten múltiples representaciones de la misma orientación (ambigüedad).

9.2. Pregunta 2: ¿Qué problema evitan los cuaterniones?

Respuesta:
Los cuaterniones evitan varios problemas críticos en robótica:

1. **Gimbal lock:** Elimina la pérdida de grado de libertad que ocurre en ángulos de Euler a ciertos ángulos.
2. **No conmutatividad:** La Actividad 2 demostró que el orden de rotaciones importa. Los cuaterniones preservan esta propiedad matemática de forma correcta mediante la multiplicación no conmutativa.
3. **Interpolación artefactada:** La interpolación lineal de Euler produce trayectorias que toman rutas inesperadas. SLERP garantiza el camino más corto.
4. **Precisión numérica:** En integraciones iterativas (como en navegación inercial), los cuaterniones acumulan menos error que Euler debido a su normalización natural.
5. **Composición de rotaciones:** Las rotaciones sucesivas con cuaterniones mantienen precisión; con Euler, los errores se amplifican.

9.3. Pregunta 3: ¿Qué relación existe entre TF2 y las transformaciones del Capítulo 2 (Corke)?

Respuesta:
TF2 es la implementación práctica y directa de los conceptos teóricos de transformaciones homogéneas de Corke:

1. **Transformaciones SE(3):** El Capítulo 2 de Corke define matrices de transformación homogéneas como:

$$T = \begin{pmatrix} R_{3\times 3} & \vec{t}_{3\times 1} \\ 0_{1\times 3} & 1 \end{pmatrix}$$

donde R es la matriz de rotación y $\vec{t}$ es el vector de traslación. TF2 implementa exactamente esta estructura en los frames.

2. **Frames y sistemas de referencia:** El árbol de transformaciones (Actividad 4) muestra la relación jerárquica entre frames. En la notación de Corke, esto es T_0^1 (transformación del frame 0 al frame 1). En ROS2, esto se expresa como la relación world → robot.

3. **Composición de transformaciones:** La no conmutatividad observada en la Actividad 2 se refleja directamente en TF2: cuando se componen transformaciones sucesivas, el orden es crítico. Si un robot tiene brazos articulados, el orden de las rotaciones determina la pose final.

4. **Representación de pose:** Una pose es la combinación de posición (traslación) + orientación (rotación). En la Actividad 3, publicamos exactamente esto: vector de posición (x, y, z) más orientación en cuaterniones.

5. **Control de orientación:** El error de orientación calculado en la Actividad 5 usa cuaterniones, que es la implementación moderna de los conceptos del Capítulo 2 en sistemas reales de control.

10. Errores comunes identificados

Durante la implementación, se identificaron los siguientes errores frecuentes:

- **Olvidar source del install:** No ejecutar `source install/setup.bash` después de compilar hace que ROS no reconozca los paquetes.
- **No normalizar cuaterniones:** En integraciones iterativas, la no normalización causa deriva numérica en la orientación con el tiempo.
- **Confusión en orden de cuaternión:** SciPy usa formato [x, y, z, w] pero algunos sistemas usan [w, x, y, z]. Esto causa invertir la orientación.
- **Fixed Frame incorrecto en RViz2:** Un nombre de frame mal escrito (ej: "World.^{en} lugar de "world") causa visualización vacía.
- **No esperar a publicación:** Suscribirse antes de que los datos se publiquen causa pérdida de mensajes iniciales.
- **Cálculo de dt incorrecto:** En la primera iteración, dt es cero o indefinido, causando rotaciones erráticas.

11. Conclusiones

1. Se implementó exitosamente un laboratorio completo de pose y cuaterniones en ROS2 Jazzy sobre Ubuntu 24.04 LTS.
2. Los cuaterniones demuestran ser la representación óptima para orientación en robótica, evitando gimbal lock e permitiendo interpolación suave y óptima.
3. La no conmutatividad de rotaciones (Actividad 2) es un concepto fundamental que afecta directamente el diseño de sistemas robóticos y el orden de operaciones.
4. TF2 es una implementación práctica y directa de los conceptos teóricos de transformaciones homogéneas del Capítulo 2 (Corke), demostrando la conexión bidireccional entre teoría y práctica.
5. SLERP proporciona trayectorias óptimas de interpolación de orientación, siendo fundamental para control suave y animación de robots.
6. La integración de conceptos matemáticos (cuaterniones, interpolación) con herramientas ROS2 (publicadores, subscribers, TF2) proporciona una base sólida para aplicaciones robóticas más complejas.

12. Referencias

1. Corke, P. (2017). *Robotics, Vision and Control: Fundamental Algorithms In MATLAB* (2nd ed.). Springer.

2. ROS2 Documentation: https://docs.ros.org/en/jazzy/

3. ROS2 TF2 Documentation: https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Tf2/Introduction-To-Tf2.html

4. SciPy Spatial Transforms: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.transform.Rotation.html

Apéndice: Evidencia en video

Se grabó video de la visualización dinámica en RViz2 mostrando el frame `robot` rotando continuamente alrededor del eje Z, demostrando la integración correcta de la orientación mediante cuaterniones y las transformaciones TF2.

Video disponible en:

[https://youtu.be/64t5pewBsx8]